

Genetic Programming

Question Bank

Optimization Techniques • CSE(DS)-5003 • Module 3

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

kanchan.rajwar@iiitbhopal.ac.in

Note to Students

This question bank is based entirely on the course notes *Genetic Programming: A Simple Introduction* distributed in class for Optimization Techniques (CSE(DS)-5003), Module 3. All definitions, examples, notation, and numerical values used in the questions are drawn directly from those notes.

The running example — symbolic regression of the target $t(x) = x^2 + 1$ using the terminal set $\{x, 1\}$, the function set $\{+, -, \times\}$, and a population of four trees — is the same example developed step by step in the notes. Students are advised to read the notes (and the companion Genetic Algorithm notes) carefully before attempting these questions.

This bank is intended for self-study and exam preparation. It is not a substitute for attending class or reading the notes.

Contents: 1. Multiple Choice Questions (20) • 2. Short Answer Questions (5) • 3. Broad Questions (5) • Answer Key.

1 Multiple Choice Questions (MCQ)

Choose the single best answer for each question.

Introduction and Basics

Q1. Genetic Programming (GP) is best described as a method that:

- (a) evolves variable-size tree programs, inspired by natural evolution
- (b) follows the gradient of a smooth function
- (c) is an exact algorithm guaranteeing the global optimum
- (d) evolves only fixed-length binary strings

Q2. In GP, a single candidate solution (the chromosome) is represented as:

- (a) a binary string
- (b) a real number
- (c) a tree
- (d) a permutation

Q3. The key difference between a GA and GP is that:

- (a) GP needs a gradient, GA does not
- (b) GP evolves variable-size programs, while GA evolves fixed-length strings

- (c) GA uses trees, GP uses strings
- (d) GP has no fitness function

Trees and Terms

Q4. A *terminal* in a GP tree is:

- (a) a leaf node — an input variable or a constant
- (b) an internal node that performs an operation
- (c) the fitness value of the tree
- (d) the depth of the tree

Q5. A *function* node in a GP tree:

- (a) is always a leaf
- (b) is a constant such as 1
- (c) performs an operation on the nodes below it
- (d) stores the decoded phenotype

Q6. Given the terminal set $\{x, 1\}$ and function set $\{+, -, \times\}$, which symbol can appear as a leaf of a tree?

- (a) +
- (b) 1
- (c) $\times$
- (d) -

Q7. Evaluating the tree for $(x \times x) + (x - 1)$ at $x = 3$ gives:

- (a) 12
- (b) 9
- (c) 8
- (d) 11

Q8. A GP tree is evaluated:

- (a) from the leaves up to the root
- (b) from the root first, then downward
- (c) strictly right to left
- (d) in a random order

Q9. The *genotype* in GP is:

- (a) the numeric output of the formula
- (b) the tree itself
- (c) the fitness value
- (d) the training data set

Sets and Initialisation

Q10. The *closure* property requires that:

- (a) all trees remain small
- (b) mutation is switched off
- (c) every function accepts any value a subtree can produce (e.g. safe division)
- (d) only the $+$ function is used

Q11. The *sufficiency* property means that:

- (a) the terminal and function sets are rich enough to express a solution
- (b) the population is large enough
- (c) enough generations are run
- (d) the mutation rate is high enough

Q12. Ramped half-and-half is a method used for:

- (a) selecting parents
- (b) creating a diverse initial population (many depths, half full and half grow)
- (c) swapping subtrees
- (d) controlling bloat

Q13. In the *full* initialisation method, all leaves lie:

- (a) at random depths
- (b) only at the root
- (c) at the same maximum depth
- (d) at depth one

Fitness

Q14. In symbolic regression, a tree's fitness is based on:

- (a) the number of nodes in the tree
- (b) the depth of the tree
- (c) how large the tree is
- (d) the total error between its output and the true data

Q15. For the target $t(x) = x^2 + 1$ (data at $x = 0, 1, 2, 3$), the tree $x \times x$ has total absolute error E equal to:

- (a) 4
- (b) 0
- (c) 8
- (d) 12

Q16. Using $F = 1/(1 + E)$, a tree that fits the data exactly ($E = 0$) has fitness:

- (a) 0

- (b) 0.5
- (c) 1
- (d) undefined

Crossover, Mutation, Bloat

Q17. Crossover in GP produces offspring by:

- (a) flipping random bits
- (b) swapping subtrees between two parents
- (c) reversing a segment of the tree
- (d) adding a constant to the root

Q18. Swapping subtrees of the parents $x + 1$ and $x \times x$ (as in the notes), the offspring that fits the data exactly is the tree for:

- (a) $x - 1$
- (b) $x + 1$
- (c) $x^2 + 1$
- (d) x

Q19. *Point mutation* in GP:

- (a) swaps two whole subtrees
- (b) changes the label of a single node (e.g. $+$ becomes $\times$)
- (c) deletes the entire tree
- (d) reverses the data set

Q20. *Bloat* in GP refers to:

- (a) the mutation rate rising over time
- (b) trees growing larger over generations with no real gain in fitness
- (c) the population size shrinking each generation
- (d) the fitness always reaching 1

2 Short Answer Questions

Answer each question in 4–6 sentences.

S1. Explain the difference between a Genetic Algorithm and Genetic Programming. Describe how a tree represents a program.

S2. Define the terminal set and the function set. Explain the closure and sufficiency properties.

S3. Describe the ramped half-and-half method for creating the initial population, and explain why diversity is important.

S4. Explain subtree crossover. Using the running example, show how crossover of $x + 1$ and $x \times x$ can discover the target $x^2 + 1$.

S5. What is bloat in GP, and what strategies are used to control it?

3 Broad Questions

Answer each question in detail, with definitions and examples where appropriate.

B1. Describe the complete working of a Genetic Programming system. Include the six-step loop, the tree representation, the role of each operator, the stopping criteria, and a worked example tracing one generation of the running example (symbolic regression of $t(x) = x^2 + 1$).

B2. Explain tree representation and fitness in GP. Define the fitness measure for symbolic regression, and give a fully worked example computing the error and fitness of the four running-example trees.

B3. Explain the full, grow, and ramped half-and-half initialisation methods in detail, with examples. Why does a diverse initial population help the search?

B4. Explain crossover and mutation in GP in detail, using the running example. Contrast the roles of the two operators.

B5. Discuss the advantages and limitations of Genetic Programming. Compare GP with the Genetic Algorithm, and state when you would prefer each.

Answer Key

MCQ Answers

Q	Topic	Ans	Q	Topic	Ans
1	Introduction	(a)	11	Sets/Init	(a)
2	Introduction	(c)	12	Sets/Init	(b)
3	Introduction	(b)	13	Sets/Init	(c)
4	Trees & terms	(a)	14	Fitness	(d)
5	Trees & terms	(c)	15	Fitness	(a)
6	Trees & terms	(b)	16	Fitness	(c)
7	Trees & terms	(d)	17	Crossover	(b)
8	Trees & terms	(a)	18	Crossover	(c)
9	Trees & terms	(b)	19	Mutation	(b)
10	Sets/Init	(c)	20	Bloat	(b)

Short Answer — Model Answers

S1. A GA evolves fixed-length strings that encode a solution (often a number). GP evolves variable-size trees that encode a program or formula. A tree stores the order of operations: internal nodes are functions, and the terminals (inputs and constants) hang below them as leaves. Evaluating the tree from the leaves up to the root computes the formula's value. For example, $(x \times x) + (x - 1)$ is a tree with $+$ at the root, and it evaluates to 11 at $x = 3$.

S2. The terminal set is the list of inputs and constants that can appear as leaves, for example $\{x, 1\}$. The function set is the list of operations that can appear as internal nodes, for example $\{+, -, \times\}$. *Closure* requires that every function accepts any value a subtree can return, so no evaluation ever fails (for instance, division is made safe by defining $a \div 0 = 1$). *Sufficiency* requires that the two sets are rich enough to express a good solution; if a necessary operation is missing, no tree can fit the target.

S3. Ramped half-and-half builds a varied first population. A range of maximum depths is chosen. At each depth, half the trees are built by the *full* method (every branch reaches that depth) and half by the *grow* method (a node may be a function or a terminal). This produces trees of many shapes and sizes. Diversity is important because a varied population explores

more of the search space and is less likely to converge early on a poor solution.

S4. In subtree crossover, one node is chosen in each parent; that node together with everything below it forms a subtree, and the two subtrees are exchanged, giving two offspring. In the running example, take the parents $x + 1$ and $x \times x$. Select the left leaf x of the first parent and the whole tree $x \times x$ of the second. Swapping them replaces that leaf with $x \times x$, producing the tree for $(x \times x) + 1 = x^2 + 1$. This offspring fits the data exactly ($E = 0$, $F = 1$), so crossover has combined two imperfect parents into the exact target.

S5. Bloat is the tendency of GP trees to grow larger from generation to generation without a matching gain in fitness. Large trees are slow to evaluate and hard to read. Bloat is controlled by setting a maximum tree depth or size, or by adding a small penalty for large trees to the fitness (called parsimony pressure). These keep the evolved programs compact.

Broad — Model Answers

B1. A GP system keeps a population of tree programs and improves them over generations using the same six-step loop as a GA. *Step 1 — Initialise:* generate random trees, e.g. $\{x + 1, x \times x, x, (x + 1) \times x\}$ (ramped half-and-half). *Step 2 — Evaluate:* run each tree on the data of $t(x) = x^2 + 1$; the total errors are $\{8, 4, 12, 4\}$ and the fitnesses $F = 1/(1 + E)$ are $\{0.111, 0.200, 0.077, 0.200\}$. *Step 3 — Select:* fitter trees ($x \times x$ and $(x + 1) \times x$) are chosen more often as parents. *Step 4 — Crossover:* swapping the left leaf x of $x + 1$ with the subtree $x \times x$ produces $(x \times x) + 1 = x^2 + 1$, which fits exactly ($E = 0$, $F = 1$). *Step 5 — Mutation:* a subtree or point mutation adds variety while the best tree is preserved. *Step 6 — Replace:* the offspring form the new population and the loop repeats. The loop stops when a tree fits the data ($E = 0$), when the best fitness stops improving, or when a maximum number of generations is reached. In this example the best fitness rose from 0.200 to 1.000 in a single generation.

B2. In GP each program is a tree: internal nodes are functions and leaves are terminals; the tree is evaluated from the leaves up to the root. For symbolic regression the fitness is $F = 1/(1 + E)$ where $E = \sum_i |g(x_i) - y_i|$ is the total absolute error of the tree's formula g against the data. Using the target $t(x) = x^2 + 1$ with data $y = 1, 2, 5, 10$ at $x = 0, 1, 2, 3$:

Tree	Formula	Total error E	Fitness F
T_1	$x + 1$	8	0.111
T_2	$x \times x$	4	0.200
T_3	x	12	0.077
T_4	$(x + 1) \times x$	4	0.200

The tree for $x^2 + 1$ has $E = 0$ and $F = 1$, the maximum possible fitness.

B3. The *full* method builds bushy trees in which every branch reaches the same maximum depth; all internal nodes are functions and all leaves lie at the bottom. The *grow* method allows each node to be either a function or a terminal (until the maximum depth forces a terminal), producing trees of varied shapes and sizes. The *ramped half-and-half* method uses a range of depth limits and, at each depth, builds half the trees by the full method and half by the grow method. This gives the widest variety of shapes and sizes. A diverse initial population is important because it spreads candidate solutions across the search space, giving crossover more useful material to combine and reducing the risk of premature convergence.

B4. *Crossover* chooses a node (and its subtree) in each of two parents and swaps the two subtrees, producing offspring that may combine good parts of both. In the running example, crossing $x + 1$ and $x \times x$ yields $(x \times x) + 1 = x^2 + 1$, which fits the data exactly. *Mutation* changes a single tree: subtree mutation replaces a chosen subtree with a new random one, and point mutation changes one node's label (for example, changing the root $+$ of $x + 1$ to $\times$ gives

$x \times 1 = x$). The roles are complementary: crossover *exploits* existing subtrees by recombining them, while mutation *explores* by introducing subtrees or labels that may be absent from the whole population. Crossover cannot create a subtree that no tree contains; mutation can.

B5. *Advantages.* GP discovers the structure of a solution, not just its numbers, returning a human-readable formula; it needs no gradient; its trees can grow or shrink, so the size of the solution need not be known in advance; and the same system fits many tasks by changing only the sets and the fitness. *Limitations.* Trees can bloat; GP usually needs large populations and many fitness evaluations, so it is slower than a GA; the function set must satisfy closure so every tree is valid; and, like all heuristics, GP gives no guarantee of optimality. *GP vs GA.* A GA evolves fixed-length strings that stand for a value; GP evolves variable-size trees that stand for a program. Prefer GP when the *form* of the solution is unknown — symbolic regression, rule discovery, evolving a controller. Prefer a GA when the solution is a fixed set of parameters or values and the encoding length is known in advance.